

آموزش شی‌گرایی (OOP) در پایتون

شی‌گرایی یا **Object-Oriented Programming** یکی از مهم‌ترین روش‌های برنامه‌نویسی است که کمک می‌کند کدهای ما منظم‌تر، قابل توسعه‌تر و قابل فهم‌تر شوند.

در شی‌گرایی، برنامه از ترکیب اشیاء (**Objects**) ساخته می‌شود. هر شیء از روی یک کلاس (**Class**) ساخته می‌شود.

در این نوت‌بوک، مفاهیم اصلی شی‌گرایی را با مثال‌های ساده و کاربردی یاد می‌گیریم.

1 کلاس و شیء

کلاس (Class) یک الگو یا نقشه برای ساخت اشیاء است.
شیء (Object) نمونه‌ای ساخته شده از روی کلاس است.

```
In [27]: class Person:
          def __init__(self, name, age):
              self.name = name
              self.age = age

          p1 = Person("Ali", 25)

          print(p1.name)
          print(p1.age)
```

Ali

25

در این مثال:

- کلاس `Person` ساخته شد
- هر شخص یک نام و سن دارد
- یک شیء از این کلاس است `p1`

2 متدها (Methods)

متدها توابعی هستند که داخل کلاس تعریف می‌شوند و رفتارهای شیء را مشخص می‌کنند.

```
In [28]: class Person:
          def __init__(self, name):
              self.name = name

          def say_hello(self):
              print(f"Hi, I'm {self.name}")
```

```
p = Person("Sara")
p.say_hello()
```

Hi, I'm Sara

ویژگی‌ها (Attributes) 3

ویژگی‌ها همان متغیرهایی هستند که به اشیاء تعلق دارند. مثلاً در کد زیر هر ماشین یک برند و یک رنگ دارد.

```
In [29]: class Car:
          def __init__(self, brand, color):
              self.brand = brand
              self.color = color

          car1 = Car("BMW", "Black")
          print(car1.brand, car1.color)
```

BMW Black

وراثت (Inheritance) 4

وراثت یعنی یک کلاس بتواند ویژگی‌ها و رفتارهای کلاس دیگر را به ارث ببرد.

```
In [30]: class Animal:
          def speak(self):
              print("Animal sound")

          class Dog(Animal):
              def speak(self):
                  print("Woof!")

          d = Dog()
          d.speak()
```

Woof!

کپسوله‌سازی (Encapsulation) 5

کپسوله‌سازی یعنی محدود کردن دسترسی مستقیم به برخی ویژگی‌ها.

```
In [31]: class BankAccount:
          def __init__(self, balance):
              # موجودی حساب بانکی را به صورت خصوصی تعریف می‌کنیم
              self.__balance = balance # private attribute

          def deposit(self, amount):
              # تابعی برای واریز وجه به حساب
              self.__balance += amount
```

```

def get_balance(self):
    # تابعی برای دریافت موجودی حساب
    return self.__balance

# استفاده از کلاس BankAccount
acc = BankAccount(1000)

# سعی در دسترسی مستقیم به موجودی حساب (ناموفق)
acc.deposit(500)

# دسترسی به موجودی حساب از طریق متد عمومی
print(acc.get_balance())

# این خط باعث خطا می‌شود
print(acc.__balance)

```

1500

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[31], line 24
     21 print(acc.get_balance())
     23 # این خط باعث خطا می‌شود
--> 24 print(acc.__balance)

AttributeError: 'BankAccount' object has no attribute '__balance'

```

6 چندریختی (Polymorphism)

چندریختی یعنی متدهایی با نام یکسان اما رفتارهای متفاوت.

```

In [32]: class Dog:
        def sound(self):
            print("Bark")

        class Cat:
            def sound(self):
                print("Meow")

        animals = [Dog(), Cat()]

        for a in animals:
            a.sound()

```

Bark
Meow

7 متدهای جادویی (Magic Methods)

متدهای خاصی هستند که با دو آندرلاین شروع و تمام می‌شوند مثل `__init__`.

```

In [33]: class Book:
        def __init__(self, title, pages):

```

```

        self.title = title
        self.pages = pages

    def __str__(self):
        return f"{self.title} - {self.pages} pages"

b = Book("Python Basics", 200)
print(b)

```

Python Basics - 200 pages

تمرین‌ها

تمرین 1

یک کلاس به نام `Student` بسازید که نام و نمره داشته باشد و متدی برای نمایش وضعیت قبولی یا مردودی داشته باشد.

تمرین 2

یک کلاس `Rectangle` بسازید که طول و عرض بگیرد و متدی برای محاسبه مساحت داشته باشد.

تمرین 3

یک کلاس `BankAccount` طراحی کنید که امکان برداشت و واریز پول داشته باشد.

تمرین 4

با استفاده از وراثت، یک کلاس `ElectricCar` از روی کلاس `Car` بسازید و ویژگی باتری به نام اضافه کنید.

تمرین 5

برنامه‌ای بنویسید که لیستی از اشیاء مختلف داشته باشد و همه آن‌ها یک متد مشترک به نام `describe()` داشته باشند (چندریختی).

 موفق باشید

پاسخ تمرین‌ها

✓ تمرین ۱ — کلاس `Student`

کلاسی می‌سازیم که نام و نمره داشته باشد و مشخص کند دانش‌آموز قبول شده یا مردود.

```

In [34]: class Student:
          def __init__(self, name, grade):
              self.name = name

```

```

        self.grade = grade

    def status(self):
        if self.grade >= 10:
            return "قبول"
        else:
            return "مردود"

s1 = Student("Ali", 15)
s2 = Student("Sara", 8)

print(s1.name, ":", s1.status())
print(s2.name, ":", s2.status())

```

Ali : قبول
Sara : مردود

تمرین ۲ — کلاس Rectangle 

محاسبه مساحت مستطیل.

In [35]:

```

class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

r = Rectangle(5, 3)
print("مساحت:", r.area())

```

مساحت: 15

تمرین ۳ — کلاس BankAccount 

حساب بانکی با قابلیت واریز و برداشت.

In [36]:

```

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("واریز انجام شد. موجودی:", self.balance)

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("برداشت انجام شد. موجودی:", self.balance)
        else:
            print("موجودی کافی نیست")

acc = BankAccount("Ali", 1000)

```

```
acc.deposit(500)
acc.withdraw(300)
acc.withdraw(2000)
```

واریز انجام شد. موجودی: 1500
برداشت انجام شد. موجودی: 1200
!موجودی کافی نیست

تمرین ۴ — وراثت Car و ElectricCar 

ماشین برقی از ماشین معمولی ارث‌بری می‌کند.

```
In [37]: class Car:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed

    def info(self):
        print(f"ماشین {self.brand} با سرعت {self.speed} km/h")

class ElectricCar(Car):
    def __init__(self, brand, speed, battery_capacity):
        super().__init__(brand, speed)
        self.battery_capacity = battery_capacity

    def battery_info(self):
        print(f"ظرفیت باتری: {self.battery_capacity} kWh")

ecar = ElectricCar("Tesla", 200, 75)
ecar.info()
ecar.battery_info()
```

km/h با سرعت 200 Tesla ماشین
kWh ظرفیت باتری: 75

تمرین ۵ — چندریختی (Polymorphism) 

چند شیء مختلف با متد مشترک describe()

```
In [38]: class Dog:
    def describe(self):
        print("من یک سگ هستم 🐶")

class Car:
    def describe(self):
        print("من یک ماشین هستم 🚗")

class Book:
    def describe(self):
        print("من یک کتاب هستم 📖")

items = [Dog(), Car(), Book()]
```

```
for item in items:  
    item.describe()
```

من یک سگ هستم 🐶
من یک ماشین هستم 🚗
من یک کتاب هستم 📖

In []: